DESIGN AND ANALYSIS OF ALGORITHMS

Emergency Ambulance Route Optimization System

A DAA 5th Semester Mini Project Report

Bachelor of Technology
Information Technology

Group: [6]

Mirant Desai | 12402080601083
Naitik Patel | 12402080601087
Nehal Savani | 12402080601092
Vivek Dobariya | 12402080601154

Faculty: [Yamini Dubey Ma'am]
Submission Date:

Certificate

This page is intentionally provided as a clean placeholder because the faculty, institution, project group and submission details were not supplied in the source project files.

Project title: Emergency Ambulance Route Optimization System

Group number: [6]

Faculty name and signature: [Yamini Dubey Ma’am]

Date:

Declaration

We, the undersigned students, will replace this placeholder with the declaration required by the institution. The current project is an academic DAA prototype developed for learning, experimentation and demonstration of shortest-path algorithms.

Student signatures: [Mirant Desai] [Naitik Patel] [Nehal Savani] [Vivek Dobariya]

Acknowledgement

We acknowledge the guidance provided through the Design and Analysis of Algorithms mini-project requirements. We also acknowledge the open technical documentation for Python, Flask, pytest, matplotlib and the Python heap module used by the implementation. The system uses a locally generated synthetic road network so that the algorithmic behavior can be inspected and reproduced without external services.

Abstract

Emergency routing is a shortest-path problem in which an ambulance must be directed from a source location to a hospital or emergency destination through a network of roads. This project presents an academic prototype that models a synthetic road network as an undirected weighted graph. Nodes represent intersections and edges represent roads whose positive weights act as the route cost. The implementation compares two shortest-path techniques: Dijkstra's Algorithm and A* Search.

The algorithms are implemented directly in Python using adjacency lists, predecessor maps, visited sets and binary min-heaps. A* uses a graph-calibrated Euclidean heuristic. The graph records the minimum observed ratio of edge cost to coordinate distance and multiplies this scale by the direct Euclidean distance to the goal. This gives A* a stronger cost-scaled lower bound while preserving admissibility under the generated coordinate-cost relationship. The Flask interface allows the user to select a scenario, source and destination, then displays the route, cost, nodes explored and measured execution time for both algorithms.

The benchmark executes seven trials for each requested graph size of 10, 50, 100 and 500 nodes, producing 56 raw measurements. The regenerated evidence shows that A* explores fewer nodes at all four sizes, while wall-clock time is mixed: it is faster at the two smaller sizes and slightly slower at the two larger sizes in this run. Both algorithms return the same route costs. The project is deliberately an offline simulation; it does not claim live traffic, GPS, real road accuracy or operational emergency-dispatch capability.

Table of Contents

1. Introduction	8
2. Problem Definition	8
3. Motivation and Real-World Application	10
4. Objectives and Scope	10
5. Background and Existing Approach	11
6. System Requirements	12
7. Proposed System and Architecture	12
8. Overall System Flow	15
9. Graph Representation	15
10. Data Structures	16
11. Dijkstra's Algorithm	17
12. A* Search	19
13. Dijkstra versus A* Comparison	21
14. Implementation Details	21
15. User Interface and Working Model	24
16. Testing	25
17. Experimental Setup and Methodology	27
18. Results and Performance Comparison	27
19. Results Interpretation	29
20. Complexity Analysis	29
21. Theoretical versus Practical Comparison	30
22. Advantages and Limitations	30
23. Future Scope	31
24. Conclusion	31
25. References	32

Page numbers correspond to this generated PDF. If the report is edited substantially, refresh the contents list before submission.

List of Figures

Figure 1. System architecture of the implemented prototype

Figure 2. Overall system flow from input to performance analysis

Figure 3. Illustrative weighted graph and shortest-path model

Figure 4. Initial dispatch interface and generated medium network

Figure 5. Computed Large-network route and comparison output

Figure 6. Average execution time versus requested input size

Figure 7. Average nodes explored versus requested input size

List of Tables

Table 1. Project objectives

Table 2. Software requirements

Table 3. Data structures used

Table 4. Dijkstra and A* comparison

Table 5. Automated test cases

Table 6. Experimental methodology

Table 7. Benchmark summary

Table 8. Complexity analysis

1. Introduction

Emergency transportation places a premium on reaching a destination through a road network with as little travel cost as possible. From an algorithmic viewpoint, the problem is not primarily about ambulance administration; it is an instance of the shortest-path problem on a weighted graph. The same abstraction can represent intersections, hospitals, emergency sites and roads without requiring a map service or a GPS device.

A graph-based model is useful because it separates the route-planning logic from the user interface. A node is a location in the network, an edge is a possible movement between two locations, and the edge weight is the cost used by the algorithm. Once this model is available, the main DAA questions become explicit: how should candidate routes be explored, how should partial costs be prioritized, how can a complete path be reconstructed, and how does the amount of work change as the graph grows?

Dijkstra's Algorithm is a natural baseline for non-negative edge weights. It expands the currently cheapest unsettled node and is guaranteed to find an optimal route. A* Search uses the same cost-so-far idea but augments it with a heuristic estimate of the remaining cost. When the heuristic is informative and admissible, A* can focus exploration toward the destination without sacrificing optimality. The practical comparison is important because fewer explored nodes do not automatically imply a lower measured runtime in a high-level language.

The objective of this project is therefore to build a small, explainable, offline prototype that demonstrates the full DAA workflow: problem definition, graph modeling, algorithm design, implementation, testing, complexity analysis, measurement and interpretation. The interface is a vehicle for showing algorithm behavior; it is not presented as a production emergency-dispatch system.

2. Problem Definition

2.1 Problem statement

Given a connected or partially connected weighted road graph, an ambulance source node and a hospital destination node, compute an optimal route under the graph's non-negative edge costs. The system should present the route and expose enough evidence to compare Dijkstra's Algorithm and A* Search on the same input.

2.2 Input

- A locally generated synthetic road network selected through the scenario control: small, medium, large or dense.
- A source node representing the ambulance location. The interface defaults to N0.
- A destination node representing the hospital or emergency destination. The interface defaults to the last node in the selected graph.
- A weighted adjacency-list graph. Each node also stores normalized x and y coordinates used by the A* heuristic.

2.3 Output

- The reconstructed path as an ordered sequence of node identifiers.
- The total route cost, or a no-path result when the destination is not reached.
- The number of explored nodes counted by the implementation.

- Measured execution time in milliseconds for the individual algorithm call.
- A side-by-side comparison and an SVG overlay of the computed route(s).

2.4 Constraints and assumptions

- Edge weights are positive in the generated graph, so both algorithms operate under non-negative costs.
- The graph is synthetic and generated locally with controlled random seeds; it is not a real geographic road network.
- The application is offline and uses no paid API, map provider, database, authentication or live traffic feed.
- The A* heuristic relies on the graph's coordinate-to-edge-cost relationship and is calibrated from observed graph edges.
- Execution time is machine-dependent and is interpreted as experimental evidence rather than a universal constant.

2.5 Objective function

The primary optimization objective is to minimize the sum of edge weights along the route. The experimental objective is to compare two correct implementations using route cost, nodes explored and execution time while preserving the same source, destination and graph.

3. Motivation and Real-World Application

In a real emergency, a route planner may need to help a response team choose among multiple roads. A shortest-path algorithm offers a principled way to reason about this choice: a road network becomes a graph, travel distance or estimated time becomes an edge cost, and the destination becomes the goal node. The algorithm can then evaluate alternatives systematically rather than relying on a manually selected route.

The educational value of this project lies in making the algorithmic trade-off visible. Dijkstra can serve as a dependable baseline because it does not need a domain-specific heuristic. A* can exploit the coordinates attached to the graph to prioritize nodes that appear closer to the destination. Comparing the two helps students connect asymptotic analysis with observed behavior: a theoretically suitable strategy still incurs data-structure and implementation costs.

Potential beneficiaries of a real system could include emergency transport coordinators, hospitals and municipal planning teams. However, this prototype should not be used for live dispatch. It has no verified road geometry, GPS, legal routing constraints, road closures, real-time traffic, safety validation or operational integration. Its value is as a transparent DAA simulation that can be extended after the core algorithms are understood.

4. Objectives and Scope

Table 1. Project objectives

No.	Objective
1	Represent a synthetic road network as a weighted adjacency-list graph with coordinates.
2	Implement Dijkstra's Algorithm from scratch using a binary min-heap and predecessor-based path reconstruction.
3	Implement A* Search from scratch using f(n) = g(n) + h(n) and a graph-calibrated Euclidean heuristic.
4	Provide an interface that accepts a scenario, source and destination and displays route metrics.
5	Test normal, boundary and failure behavior through automated tests.
6	Measure and compare the algorithms across multiple graph sizes using repeated trials and saved evidence.

The scope intentionally excludes live maps, real traffic, user accounts, databases, multi-ambulance scheduling and commercial routing APIs. These are possible future directions, not current features.

5. Background and Existing Approach

5.1 Weighted shortest-path problem

In a weighted graph G = (V, E), each edge has a cost. The shortest-path problem asks for a minimum-cost sequence of connected edges from a source s to a destination t. When all weights are non-negative, a label-setting method can safely finalize a node when no cheaper frontier route can reach it.

5.2 Dijkstra as a baseline

Dijkstra's Algorithm repeatedly selects the frontier node with the smallest tentative distance. It relaxes each outgoing edge: if reaching a neighbor through the current node produces a lower cost, the neighbor's distance and predecessor are updated. A binary min-heap makes the minimum extraction efficient. In this project, the search stops when the destination is removed from the heap and marked visited.

5.3 A* as heuristic search

A* changes the priority from cost-so-far to an estimated total cost, f(n) = g(n) + h(n). Here g(n) is the known cost from the source to n, while h(n) estimates the remaining cost to the destination. A useful heuristic can reduce unnecessary expansion, but it must be designed carefully when optimality is required.

5.4 Limitations of simple approaches

A purely visual or manually selected route has no explicit optimality argument and does not scale as the graph grows. Dijkstra is systematic but may explore many nodes that are not directionally useful. A* adds direction, but the heuristic calculation adds work and its benefit depends on graph structure. The current project compares these effects on controlled synthetic data rather than claiming general superiority.

6. System Requirements

6.1 Hardware requirements

The project is a lightweight local prototype. The following are typical requirements rather than a claim about a required production configuration: a modern dual-core CPU, 4 GB or more of RAM, approximately 200 MB of free storage for the source, dependencies and evidence, and a display capable of viewing a local web page.

6.2 Software requirements

Table 2. Software requirements

Technology	Observed version/context	Role
Python	3.12.10 in the local project environment	Application, graph model, algorithms and benchmark scripts
Flask	3.0.3	Local HTTP server and JSON/application routes
HTML/CSS/JavaScript	Browser platform	Dashboard controls, SVG graph and result cards
pytest	8.3.3	Automated algorithm and graph tests
matplotlib	3.9.2	Benchmark performance charts
heapq / time / pathlib / csv / json	Python standard library	Priority queues, timing, files and benchmark evidence

The application is installed from requirements.txt and runs with `python app.py`. The benchmark is run with `python benchmarking/benchmark.py`; tests are run with `pytest -q`.

7. Proposed System and Architecture

The proposed system has a thin web layer around a deliberately visible algorithmic core. The browser requests a synthetic network or submits a source/destination/scenario request. Flask builds the graph and invokes the two Python functions. The returned route and metrics are serialized as JSON, after which the browser draws the network and overlays.

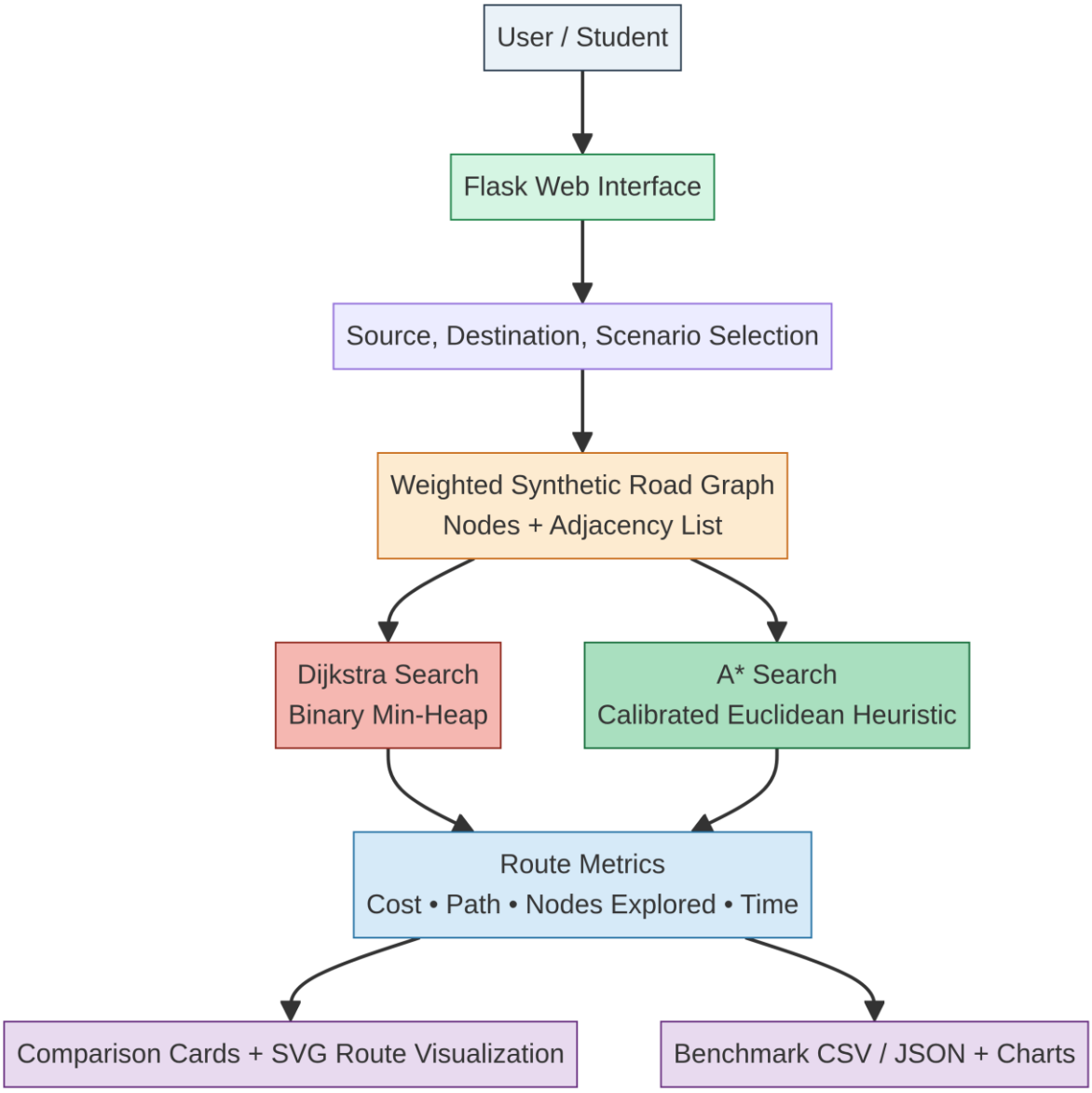


Figure 1. System architecture of the implemented prototype

The architecture keeps the graph and shortest-path logic independent from the visual presentation. Benchmarking reuses the same algorithm modules, which helps ensure that the measured code and the interactive code follow the same route logic.

7.1 Module responsibilities

- `app.py`: Flask entry point, network endpoint, route-comparison endpoint and evidence-file route.
- `graph/graph.py`: Node record, adjacency list, edge insertion, coordinate distance, calibration scale and heuristic.
- `graph/data.py`: deterministic synthetic grid generation and scenario selection.
- `algorithms/dijkstra.py`: from-scratch Dijkstra search, path reconstruction and metrics.
- `algorithms/astar.py`: from-scratch A* search, path reconstruction and metrics.
- `benchmarking/benchmark.py`: repeated measurements, CSV/JSON output and matplotlib charts.
- `templates/` and `static/`: dashboard markup, style and SVG rendering logic.
- `tests/`: automated behavior and heuristic checks.

8. Overall System Flow

The application follows the same conceptual sequence as the DAA project requirements: define the problem, obtain a graph and endpoints, execute both algorithms, calculate metrics, compare the results and visualize the evidence. The flow below groups the UI and algorithm actions without adding behavior that is not present in the implementation.

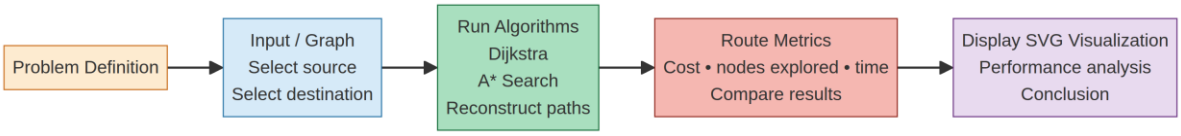


Figure 2. Overall system flow from input to performance analysis

The interactive route request and the offline benchmark share the graph-generation and algorithm modules. The benchmark adds repeated trials and file-based evidence, whereas the interface adds SVG visualization and side-by-side metric cards.

9. Graph Representation

9.1 Vertices and edges

The `Node` data class stores an identifier, normalized x and y coordinates, and a human-readable label. The `RoadGraph` object stores nodes in a dictionary and neighbors in an adjacency dictionary. An undirected road is inserted in both directions. Each adjacency entry is a pair `(neighbor, cost)`.

9.2 Synthetic grid generation

`generate_grid_graph(size, traffic_factor, seed)` chooses a grid side large enough for the requested number of nodes, inserts nodes row by row and connects horizontal and vertical neighbors. Some local diagonal edges are added probabilistically to create alternate routes. Horizontal and vertical edge costs are between 1.05 and 1.35 before the traffic factor; diagonal costs are between 1.35 and 1.60 before the factor. The `dense` scenario uses a simulated traffic factor of 0.82 and a different seed. This is synthetic data, not a claim about live traffic.

9.3 Illustrative graph model

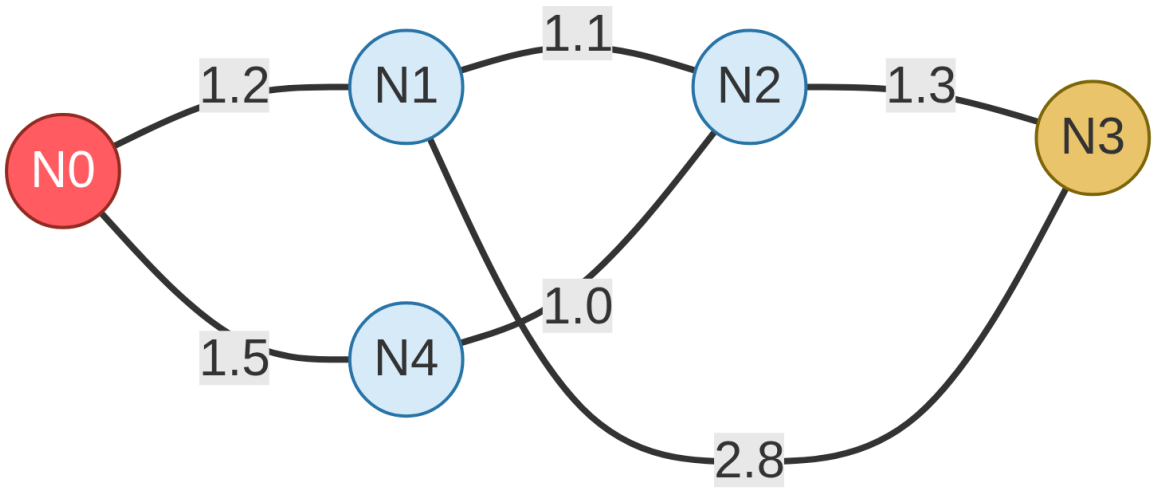


Figure 3. Illustrative weighted graph and shortest-path model

This small diagram is an explanatory model of the same concepts used by the implementation: node identifiers, positive edge weights, a source, a goal and multiple possible routes. The application itself generates larger grid networks locally.

9.4 Path reconstruction

Both algorithms maintain a predecessor map. When a relaxation improves a node's best-known cost, the current node is recorded as its predecessor. Once the destination is reached, reconstruction follows predecessor links backward to the source and reverses the collected sequence. If the destination was never reached, the returned path is empty and `found` is false.

10. Data Structures

Table 3. Data structures used

Structure	Purpose in code	Algorithmic contribution
Node data class	Stores id, x, y and label for each location.	Provides graph identity, heuristic coordinates and readable UI metadata.
Adjacency list	Maps each node to `(neighbor, weight)` pairs.	Stores sparse road connectivity and supports edge relaxation.
Distance / g-cost dictionary	Dijkstra uses `distances`; A* uses `g_cost`.	Records the best known source-to-node cost.
Predecessor dictionary	Maps each improved node to its prior node.	Supports final route reconstruction.
Binary min-heap	Stores `(priority, cost, node)` tuples.	Returns the next lowest-priority frontier entry.
Visited set	Tracks nodes whose heap entry has been finalized.	Prevents repeated expansion of the same finalized node.
CSV / JSON result records	Stores raw trials and summary averages.	Makes the experiment inspectable and reproducible.

11. Dijkstra's Algorithm

11.1 Introduction and working principle

Dijkstra's Algorithm is used here as the non-heuristic baseline. It begins with distance zero at the source and infinity at all other nodes. The source is pushed onto a min-heap. At each iteration, the lowest-cost unvisited node is removed, its outgoing roads are relaxed, and improved neighbors are added to the heap. If the removed node is the destination, the loop stops because no later frontier entry can have a lower non-negative cost.

11.2 Algorithm steps

1. Initialize all distances to infinity and set the source distance to zero.
2. Create a predecessor map with the source predecessor set to None.
3. Push `(0, source)` into the min-heap.
4. Remove the lowest-cost heap entry; skip it if the node was already visited.
5. Mark the node visited and increment the explored-node counter.
6. If it is the destination, stop the search.
7. For each neighbor, calculate the candidate cost through the current node.
8. If the candidate improves the neighbor's distance, update its distance, predecessor and heap entry.
9. Reconstruct the path from the destination when the loop finishes.

11.3 Pseudocode

```
DIJKSTRA(graph, source, goal)
    distance[node] <- infinity for every node
    distance[source] <- 0
    parent[source] <- None
    heap <- [(0, source)]
    visited <- empty set

    while heap is not empty
        (cost, current) <- extract minimum
        if current in visited: continue
        add current to visited
        explored <- explored + 1
        if current = goal: break

        for (neighbor, weight) in adjacency[current]
            candidate <- cost + weight
            if candidate < distance[neighbor]
                distance[neighbor] <- candidate
                parent[neighbor] <- current
                insert (candidate, neighbor) into heap

    return reconstruct(parent, source, goal), distance[goal], explored
```

11.4 Data structures and reconstruction

The implementation's `distances`, `parents`, `visited` and `queue` correspond directly to the pseudocode. The helper `_reconstruct` handles the source-equals-goal case, the normal route case and the no-path case without calling an external shortest-path library.

11.5 Complexity and limitations

With an adjacency list and binary min-heap, the standard worst-case bound is O((V + E) log V). The graph itself uses O(V + E) storage. The auxiliary search structures use O(V) dictionary/set/heap-scale storage in the usual sparse-graph accounting, so total memory including the input graph is O(V + E). Dijkstra's limitation is not correctness but direction: it does not use geometric information to prefer nodes that appear closer to the goal.

11.6 Small worked interpretation

In the illustrative graph, Dijkstra may first compare the direct-looking branch with the lower-cost multi-edge branch. It does not decide based on geometric closeness; it decides only from accumulated cost. If a neighbor can be reached more cheaply through a newly expanded node, its predecessor is replaced. That replacement behavior is the relaxation step that ultimately determines the reconstructed path.

12. A* Search

12.1 Introduction and working principle

A* Search retains Dijkstra's non-negative-cost relaxation idea but ranks a frontier entry with an estimated total cost. The implementation stores a tuple `(f, g, node)`, where g is the cost from the source to the node and f is g plus the heuristic estimate to the goal. The destination is still accepted when it is removed from the priority queue.

12.2 f(n) = g(n) + h(n)

- `g(n)`: the best known cost from the source to node n.
- `h(n)`: an estimate of the remaining cost from n to the destination.
- `f(n)`: the priority used to select the next frontier entry.

12.3 Heuristic used in this implementation

The graph stores normalized coordinates for every node. When an edge is added, the graph computes its coordinate length and observes the ratio `edge_cost / coordinate_distance`. The smallest positive ratio over all edges becomes `heuristic_scale`. The heuristic is then:

```
h(a, b) = heuristic_scale * EuclideanDistance(a, b)
```

This calibration is important because raw coordinate distances are much smaller than the graph's edge-cost units. Multiplying by the minimum observed cost-per-coordinate-unit puts the estimate on the same scale as route costs.

12.4 Why the heuristic is admissible here

For every generated edge (u, v), the calibration scale is no larger than `edge_cost(u,v) / coordinate_distance(u,v)`. Therefore, each edge cost is at least the scale multiplied by its coordinate length. For any multi-edge route, the sum of coordinate lengths is at least the direct Euclidean distance between the endpoints by the triangle inequality. Consequently, the scaled direct Euclidean distance cannot exceed the cost of any available route. This is the lower-bound property required for admissibility in this generated model.

12.5 Pseudocode

```
A_STAR(graph, source, goal)
    g_cost[node] <- infinity for every node
    g_cost[source] <- 0
    parent[source] <- None
    heap <- [(heuristic(source, goal), 0, source)]
    visited <- empty set

    while heap is not empty
        (f, cost, current) <- extract minimum
        if current in visited: continue
        add current to visited
        explored <- explored + 1
        if current = goal: break

        for (neighbor, weight) in adjacency[current]
            candidate <- cost + weight
            if candidate < g_cost[neighbor]
                g_cost[neighbor] <- candidate
                parent[neighbor] <- current
```

```
                priority <- candidate + heuristic(neighbor, goal)
                insert (priority, candidate, neighbor) into heap

    return reconstruct(parent, source, goal), g_cost[goal], explored
```

12.6 Advantages and limitations

The calibrated heuristic is more informative than raw normalized distance and, in the benchmark, reduces the number of explored nodes at every tested input size. It does not guarantee lower wall-clock time because each expansion computes a heuristic and manages heap tuples. A* is therefore goal-directed but not universally faster. Its optimality depends on the heuristic remaining a lower bound; the implementation's documented guarantee applies to the generated graph model and its cost-coordinate relationship.

13. Dijkstra versus A* Comparison

Table 4. Dijkstra and A* comparison

Criterion	Dijkstra	A*
Search strategy	Cost-so-far expansion	Estimated total-cost expansion
Heuristic	None	Graph-calibrated Euclidean lower bound
Priority	g(n)	f(n) = g(n) + h(n)
Goal direction	Indirect; explores by cost	Directs priority toward destination
Optimality condition	Non-negative edge weights	Admissible and consistent heuristic + non-negative edge costs
Core structures	Adjacency list, heap, distances, parents, visited	Same, plus coordinate heuristic
Route cost in benchmark	Matches A* at all sizes	Matches Dijkstra at all sizes
Nodes explored	Higher in all four summaries	Lower in all four summaries
Execution time	Faster at sizes 100 and 500 in this run	Faster at sizes 10 and 50 in this run
Practical behavior	Reliable baseline; can search broadly	Goal-directed; heuristic work adds overhead

The comparison is not a contest with a universal winner. Dijkstra is simpler and has no heuristic-calibration dependency. A* can avoid expanding nodes that are obviously away from the destination, but its heap priority and heuristic calculations are additional work. The benchmark demonstrates the distinction between search effort and wall-clock time rather than hiding it.

14. Implementation Details

14.1 Backend and frontend

The backend is a compact Flask application. `GET /api/network` returns the selected graph in JSON form. `POST /api/route` builds the selected graph, validates the source and destination, invokes Dijkstra and A*, and returns the network plus both results. The `/results/<filename>` route serves generated evidence files stored in the root `results/` directory. The frontend is a single HTML page with CSS panels and JavaScript controls. SVG line and circle elements display the network, source, destination and routes.

14.2 Project structure

```
Emergency-Ambulance-Route-Optimization/
├── app.py
├── requirements.txt
├── README.md
├── VIVA_QA.md
├── algorithms/
│   ├── dijkstra.py
│   └── astar.py
├── graph/
│   ├── graph.py
│   └── data.py
├── benchmarking/
│   └── benchmark.py
├── data/
│   └── road_network.json
├── templates/
│   └── index.html
├── static/
│   ├── css/style.css
```

```
│   └── js/app.js
├── tests/
│   └── test_algorithms.py
├── results/
│   ├── benchmark_results.csv
│   ├── benchmark_summary.json
│   ├── performance_time.png
│   └── performance_nodes.png
├── screenshots/
└── report/
```

14.3 Benchmark implementation

The benchmark loops over sizes `(10, 50, 100, 500)`, creates a graph with seed `100 + trial_index`, runs each algorithm from N0 to the last generated node, records time, explored nodes, route cost and path length, and writes the raw rows to CSV. It then averages the rows by input size and algorithm, writes JSON methodology plus summary data, and plots the two required charts with matplotlib.

15. User Interface and Working Model

15.1 Initial interface

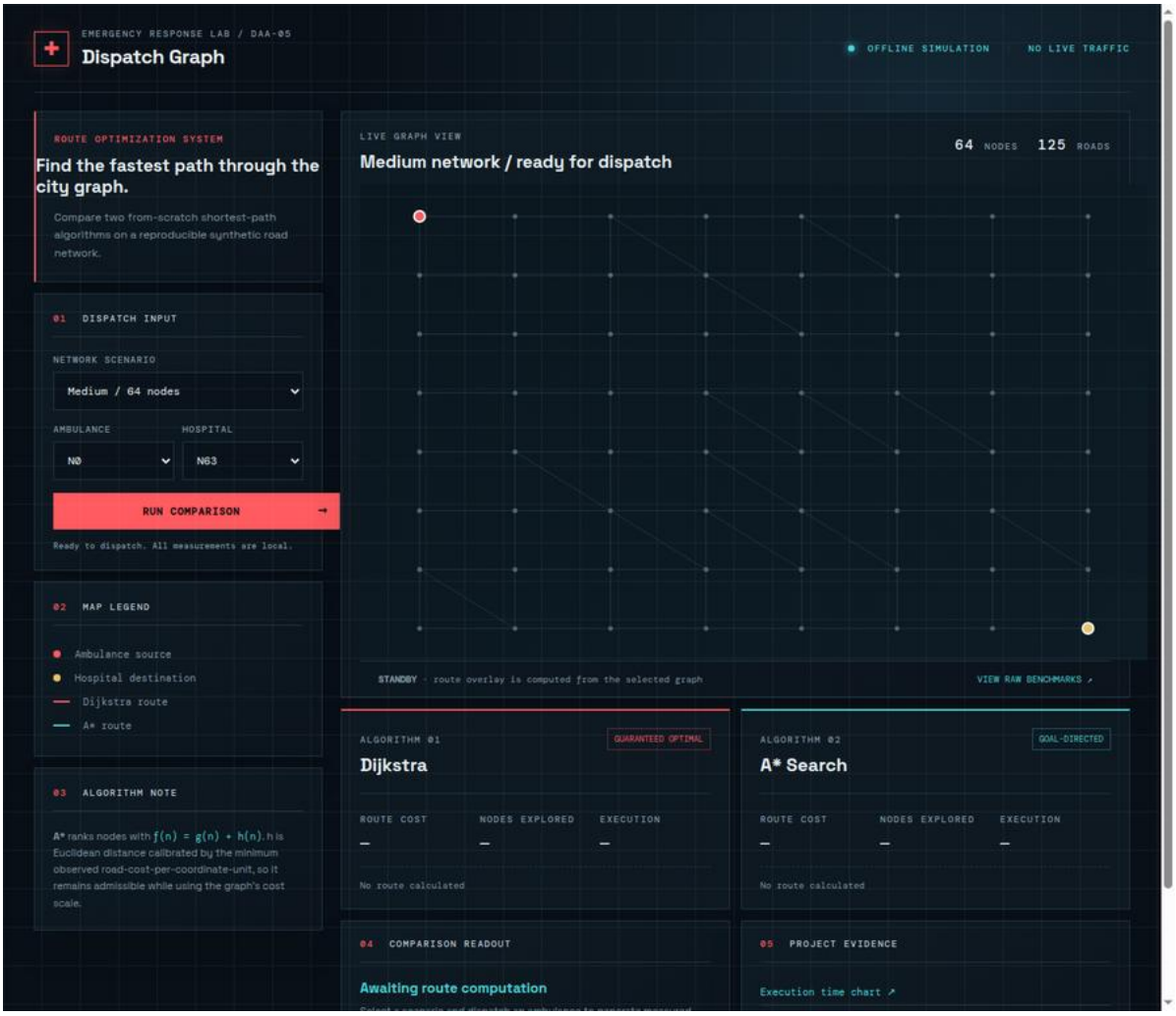


Figure 4. Initial dispatch interface and generated medium network

This actual project screenshot shows the scenario selector, source and hospital controls, map legend, heuristic note and the generated medium graph. The map contains the red ambulance source marker at N0 and the yellow destination marker at N63 before a route comparison is run.

15.2 Computed route and result cards

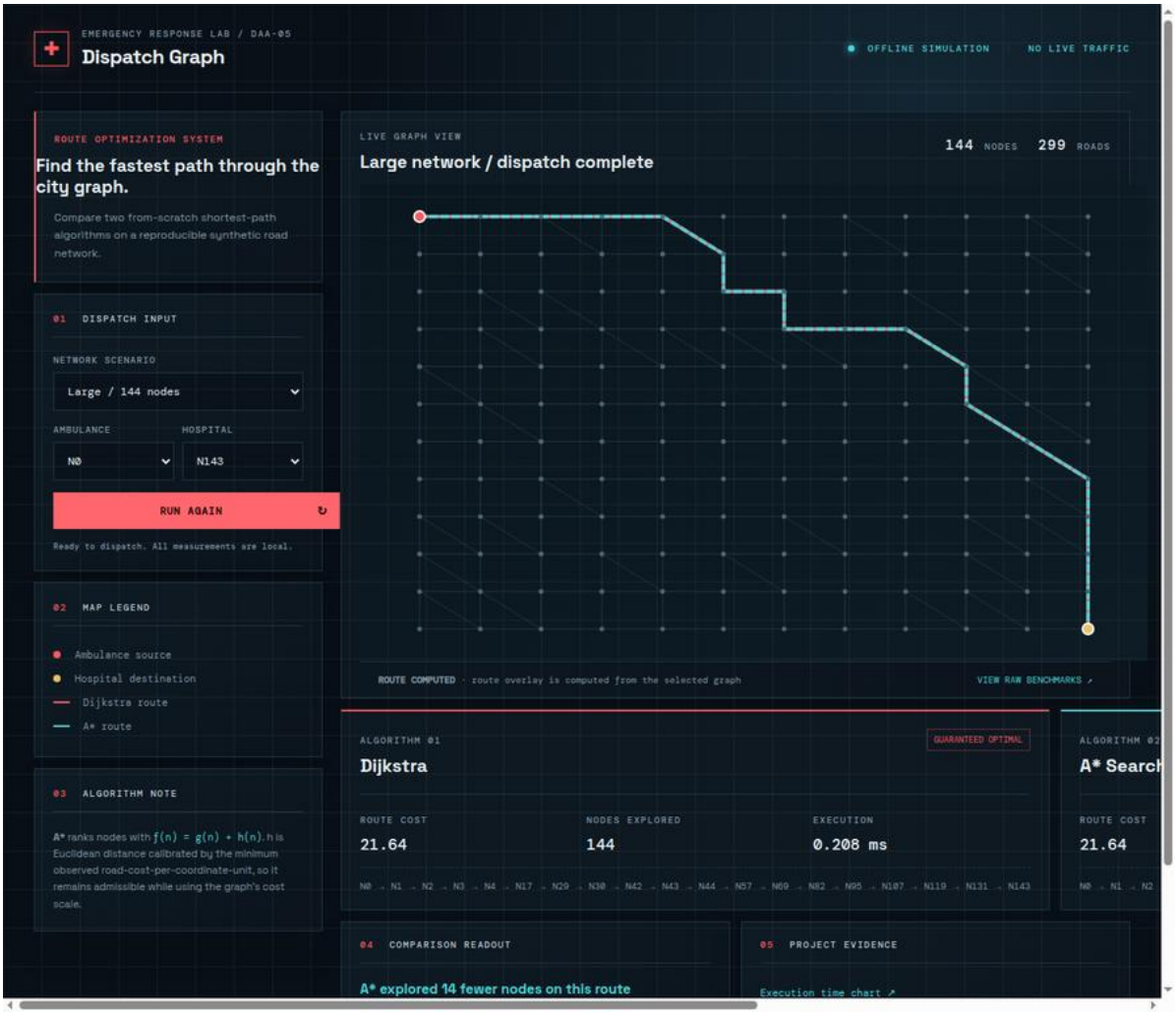


Figure 5. Computed Large-network route and comparison output

This screenshot is from the working application after running the Large / 144-node scenario. It shows the route overlay, route status, Dijkstra result card and the A* comparison card. The captured example reports a route cost of 21.64 for both algorithms, with 144 versus 130 explored nodes; execution time is shown for that individual UI call.

The UI is intentionally evidence-oriented. It does not pretend to be a geographic map: the SVG uses the graph's normalized coordinates, and the labels explicitly identify the experience as an offline simulation with no live traffic. The application preserves the red solid Dijkstra route under the thinner cyan dashed A* route when both paths coincide.

Sample execution: For the displayed Large / 144-node example, the input is ambulance N32 and hospital N84. Both algorithms return route cost 21.64; Dijkstra finalizes 144 nodes while A* finalizes 130 nodes. The execution-time values are machine-dependent and are shown directly in the captured working-model screen.

16. Testing

The automated tests are located in `tests/test_algorithms.py`. They validate route cost consistency, the source-equals-goal boundary case, no-path behavior, path-edge validity, heuristic admissibility on every observed edge, and the fact that the calibrated heuristic uses the graph's cost scale.

Table 5. Automated test cases

ID	Input	Expected result	Actual result	Status
TC-01	25-node graph, N0 → N24	Both algorithms find a route with equal cost.	Both found a route; costs equal within 1e-9.	Pass
TC-02	9-node graph, N4 → N4	Path is [N4] for both algorithms.	Both returned [N4].	Pass
TC-03	4-node graph with N0/N1 disconnected	Both algorithms report no path.	Both returned found=False.	Pass
TC-04	36-node graph, A* N0 → N35	Every consecutive path pair is an existing graph edge.	All path edges validated.	Pass
TC-05	64-node graph, every observed edge	h(u,v) ≤ edge weight for every edge.	All admissibility assertions passed.	Pass
TC-06	64-node graph, N0 → N63	Calibrated scale > 1 and h exceeds raw normalized distance.	Both assertions passed.	Pass

16.1 Edge-case discussion

- Source equals destination is handled by immediate path reconstruction with a one-node path.
- No-path behavior is tested by disconnecting nodes in the graph adjacency structure.
- Multiple possible paths are part of the grid/diagonal graph design; relaxation chooses the minimum-cost predecessor.
- Path validity is checked by confirming each consecutive pair is an adjacency-list edge.
- Cost consistency is checked by comparing Dijkstra and A* on the same graph and endpoints.

17. Experimental Setup and Methodology

Table 6. Experimental methodology

Item	Current experiment
Dataset	Deterministic synthetic grid graphs generated locally by graph/data.py
Requested sizes	10, 50, 100 and 500 nodes
Trials	7 trials per size per algorithm; 56 raw measurements
Seed policy	Seed = 100 + trial index in benchmark.py
Endpoints	N0 to the last generated node for each graph
Measured metrics	time_ms, nodes_explored, route_cost, path_length
Timing method	time.perf_counter() around each algorithm call
Outputs	results/benchmark_results.csv and benchmark_summary.json
Charts	performance_time.png and performance_nodes.png generated by matplotlib
Road-network conditions	The UI supports small, medium, large and dense scenarios. The dense scenario applies a simulated traffic factor; the quantitative benchmark below isolates input-size scaling and does not claim a full traffic-condition benchmark.
Environment	Python 3.12.10; Flask 3.0.3; pytest 8.3.3; matplotlib 3.9.2

The benchmark does not claim a controlled operating-system scheduler or hardware laboratory. It is a reproducible local measurement script. Repeated trials reduce the influence of a single noisy timing sample, but the measured milliseconds remain dependent on the machine and current process load.

18. Results and Performance Comparison

Table 7. Benchmark summary from 56 actual measurements

Input size	Dijkstra avg ms	A* avg ms	Dijkstra nodes	A* nodes	Avg route cost (both)
10	0.0120	0.0111	7.29	4.00	3.0393
50	0.0440	0.0305	33.43	12.86	7.6555
100	0.1125	0.1270	100.00	64.14	16.2637
500	0.6367	0.6420	482.29	295.57	34.3338

For every tested input size, the Dijkstra and A* averages have the same route cost. This is the expected correctness relationship for the tested graphs. The nodes-explored comparison favors A*: its averages are lower at 10, 50, 100 and 500 nodes. The time comparison is more nuanced, as shown in the next figures and interpretation section.

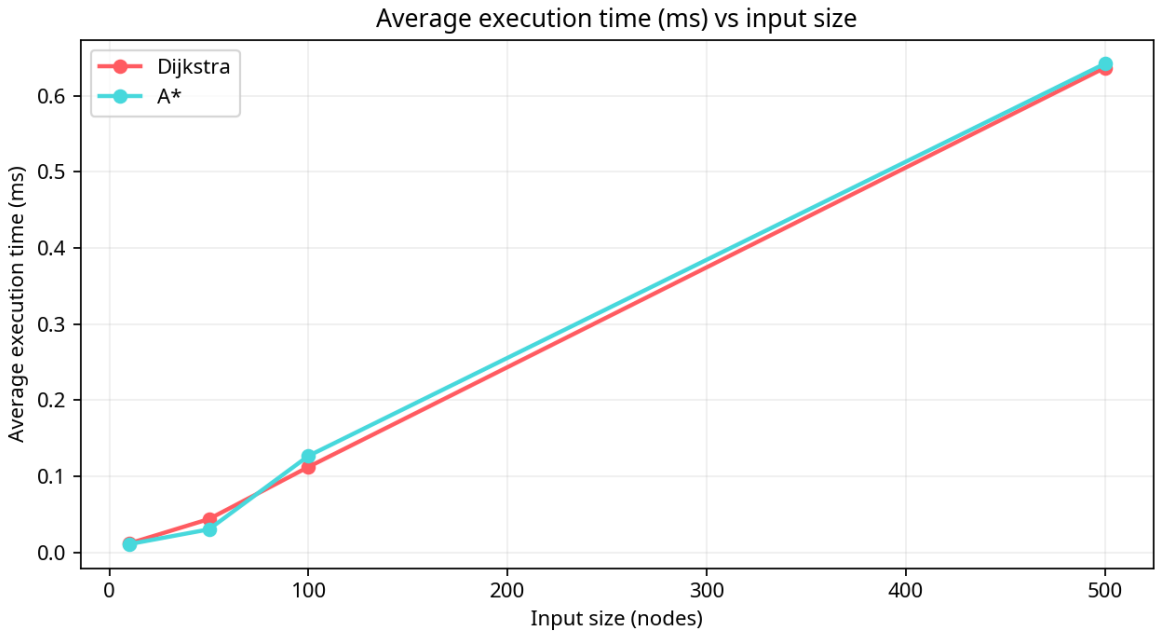


Figure 6. Average execution time versus requested input size

The execution-time chart rises with graph size for both algorithms. In this run A* is slightly faster at 10 and 50 nodes, while Dijkstra is slightly faster at 100 and 500 nodes. The close curves reinforce that fewer expansions do not automatically produce lower wall-clock time.

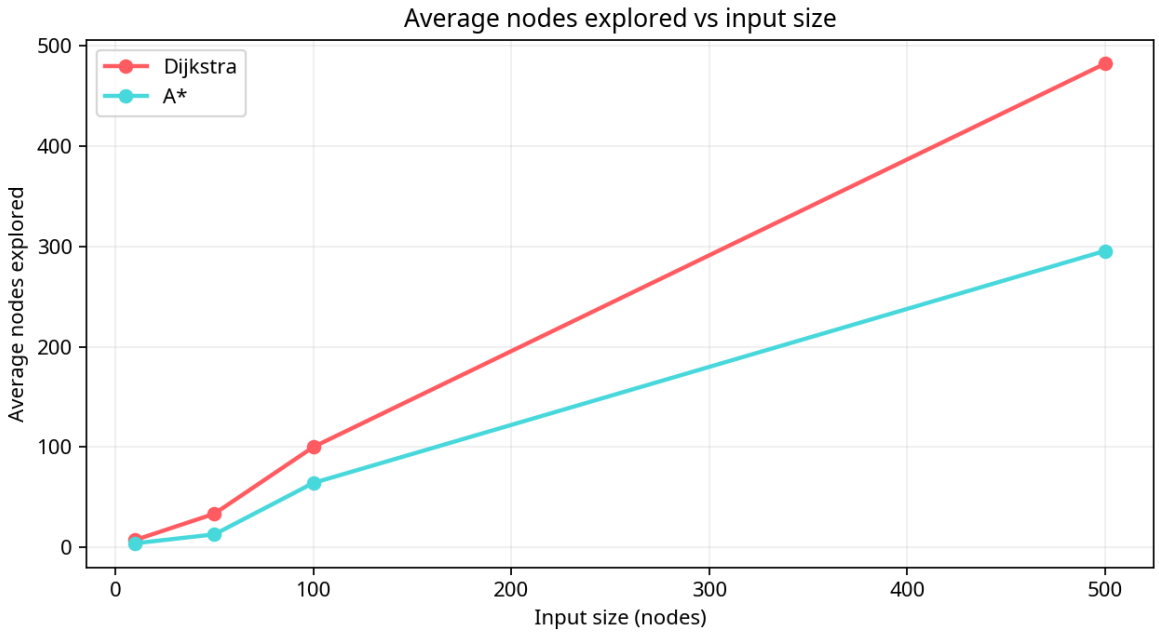


Figure 7. Average nodes explored versus requested input size

The nodes-explored chart shows the stronger directional benefit of the calibrated A* heuristic. A* explores 4.00 versus 7.29 nodes at size 10, 12.86 versus 33.43 at size 50, 64.14 versus 100.00 at size 100, and 295.57 versus 482.29 at size 500.

19. Results Interpretation

19.1 Effect of input size

As the requested graph size increases, both average execution time and average explored-node counts generally increase. Dijkstra reaches 482.29 average explored nodes at size 500, while A* reaches 295.57. The growth is consistent with the fact that larger grids expose more possible frontier nodes and require more heap operations.

19.2 Effect of heuristic guidance

The calibrated heuristic changes the search order so that nodes with a promising total estimated cost are considered earlier. The result is fewer finalized expansions in every measured size. This is the clearest practical benefit visible in the current experiment.

19.3 Fewer nodes versus lower time

Fewer explored nodes do not necessarily mean a lower measured runtime. Each A* expansion computes a coordinate distance and a calibrated heuristic, and both algorithms still perform heap pushes, tuple comparisons and dictionary operations. At small sizes, the implementation overhead is comparable to the entire search. At larger sizes, the observed A* time is slightly above Dijkstra even though A* explores fewer nodes. The correct conclusion is therefore conditional: A* is more goal-directed in this dataset, but neither algorithm is universally faster.

19.4 Machine-dependent timing

The benchmark uses `time.perf_counter()` and reports averages, but it does not isolate CPU frequency changes, background processes, interpreter effects or operating-system scheduling. Timing values should be used to compare this run's trend, not as a universal performance guarantee.

20. Complexity Analysis

Table 8. Complexity analysis

Algorithm	Best case	Average case	Worst case	Space complexity
Dijkstra	O(1) when source = goal; otherwise input-dependent	Input-dependent	O((V+E) log V)	O(V) auxiliary; O(V+E) including graph
A*	O(1) when source = goal; otherwise input-dependent	Input/heuristic-dependent	O((V+E) log V)	O(V) auxiliary; O(V+E) including graph

Here V denotes the number of vertices and E the number of undirected road connections represented in the adjacency list. A binary heap gives logarithmic insertion and removal in the number of heap entries. The adjacency list takes O(V + E) graph storage. The dictionaries and visited set scale with O(V); the heap can hold multiple entries for a node because the code does not perform decrease-key in place, but the standard combined bound remains O((V + E) log V) for this implementation style.

Best-case wording requires care: there is no single universal average-case complexity for arbitrary graph instances. The source-equals-goal case is explicitly O(1) in the implementation's control flow. For other inputs, practical work depends on graph connectivity, edge weights, destination location and the amount of frontier exploration.

21. Theoretical versus Practical Comparison

Asymptotic complexity describes how resource use scales under a chosen representation; it does not predict the exact millisecond value of a particular Python process. Two implementations with the same asymptotic upper bound can differ in constants, branch behavior, heap contents and number of expanded nodes.

- Graph structure affects how many alternative frontier entries are generated and how quickly the destination becomes competitive.
- Heuristic quality determines whether A* receives useful direction or behaves similarly to cost-only exploration.
- Explored-node count is a structural measure, while execution time includes Python interpreter and data-structure overhead.
- Heap operations matter because the implementation may push multiple improved entries rather than updating a heap item in place.
- Small input sizes make timing noise and fixed overhead a large fraction of the measurement.
- The benchmark's seven-trial average reduces but does not eliminate machine-dependent variation.

22. Advantages and Limitations

22.1 Advantages

- The two main algorithms are implemented from scratch and remain easy to trace in a viva.
- The graph is deterministic and self-contained, making the experiment runnable without internet access or API keys.
- The same route metrics are visible in the UI and saved in benchmark evidence.
- The graph-calibrated A* heuristic is stronger than raw normalized distance while remaining justified for the generated model.
- Automated tests cover correctness, failure behavior, path validity and heuristic properties.

22.2 Limitations

- The network is synthetic and does not reproduce a verified geographic road system.
- Edge weights are simulated costs rather than live travel time or measured distance.
- There is no GPS, real-time traffic, road closure model, turn restriction or emergency-priority policy.
- The interface is a local Flask prototype rather than a production dispatch application.
- The benchmark uses a small set of controlled graph sizes and one algorithmic workload pattern.

23. Future Scope

Future work should be separated from the current implementation. A later version could import a verified road network, convert road distance or travel-time estimates into edge costs, and model time-dependent traffic or road closures. A geographic front end could display the route against real coordinates, but that would require careful data licensing, validation and safety review.

Additional algorithmic extensions could compare bidirectional search, multi-source routing, multi-criteria costs or dynamic rerouting. A multi-ambulance system would introduce an allocation and scheduling problem beyond the current single-source/single-destination shortest-path task. These are opportunities for future DAA or systems projects rather than claims about the current prototype.

24. Conclusion

This project formulates emergency ambulance routing as a weighted shortest-path problem on a synthetic road network. The graph uses nodes, positive edge weights, coordinates and an adjacency list. Dijkstra's Algorithm provides a cost-only baseline, while A* adds a graph-calibrated Euclidean lower bound to direct the search toward the destination. Both implementations use explicit heaps, cost maps, predecessor maps and visited sets, so the algorithmic work is visible rather than delegated to a library shortest-path function.

The Flask prototype connects the algorithms to a usable local interface. It lets a user select a scenario, source and destination, view the generated network and inspect route cost, explored nodes, execution time and the reconstructed path. The report's screenshots and diagrams are from the working project, and the benchmark outputs are generated from the current code and stored as raw CSV, summary JSON and charts.

The measured results show a consistent reduction in explored nodes for A* across the tested sizes, with equal route costs in all summaries. They also show why experimental interpretation must remain honest: A* is not always faster in wall-clock time because heuristic computation and heap behavior contribute overhead. The final result is therefore a coherent DAA mini-project demonstration of problem modeling, algorithm design, implementation, testing, complexity analysis and evidence-based comparison, not a claim of production emergency-routing accuracy.

25. References

1. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271.

2. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

3. Python Software Foundation. heapq — Heap queue algorithm. Python Documentation. https://docs.python.org/3/library/heapq.html

4. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/

5. pytest Development Team. pytest Documentation. https://docs.pytest.org/

6. Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/

The synthetic graph data, benchmark CSV/JSON and screenshots cited in this report are project-generated artifacts stored in the submitted project directory. No external road dataset or live map service was used.